

Injaaz Application — Overview

This document describes the Injaaz web application at a high level: purpose, architecture, major modules, and how they relate. For setup and deployment, see README.md, SETUP.md, and the guides under docs/.

1. Purpose

Injaaz is a Flask-based platform for facilities and operations work: digital forms, inspections, review workflows, reporting, document handling, and administration. It is built as a modular monolith—one application with many feature blueprints sharing authentication, database, and infrastructure.

2. Technical foundation

Area	Technology
Runtime	Python, Flask
API authentication	JWT (Flask-JWT-Extended); tokens via Authorization header and optionally cookies
Data	SQLAlchemy, Flask-Migrate; SQLite (typical dev), PostgreSQL (production)
Security	CSRF on form routes where applicable; API routes often JWT-exempt; rate limiting with Redis when configured; standard security headers
Files / media	Local upload directories; Cloudinary for production asset hosting where configured
Email	Mailjet (e.g. MMR scheduled reports) via shared email helpers
Frontend	Server-rendered HTML, CSS, JavaScript; PWA-related assets (manifest, offline page); optional Capacitor for mobile shells
Production entry	wsgi.py; deployment documented for platforms such as Render

3. Core platform

These capabilities apply across the product:

Area	Role
Authentication (/api/auth)	Login, registration, JWT access/refresh
Main UI	Routes such as /login, /dashboard, /about
Admin (/api/admin)	User management, access control, admin-only pages (e.g. devices, BD admin shell)
Workflow (/api/workflow)	Submission pipelines, pending reviews, submitted forms, supervisor/reviewer flows
Reports API (/api/reports)	On-demand report regeneration where implemented

4. Operational and business modules

Module	URL prefix (typical)	Role
HVAC / MEP	/hvac-mep	HVAC & MEP forms, generators, reporting
Civil	/civil	Civil works forms and reports
Cleaning	/cleaning	Cleaning services forms and reports
Inspection	/inspection	Inspection workflows spanning relevant trades
HR	/hr	Human resources screens and APIs
Procurement	/procurement	Procurement workflows
MMR (reports)	/admin/mmr	CAFM Excel upload, analytics dashboard, Excel report generation, chargeable/non-chargeable rules, scheduled daily email (scheduler uses Asia/Dubai by default), optional network save paths
Business Development	/bd, /admin/bd	BD-focused flows and admin entry
DocHub	/api/docs, /dochub	Document hub for authorized users

Shared code lives under `app/`, `common/`, and domain folders such as `module_hvac_mep/`, `module_civil/`, etc.

5. How the pieces fit together

1. Users authenticate and use the dashboard; permissions gate admin, workflow, and module routes.
2. Field modules (HVAC, civil, cleaning, inspection) capture structured data and attachments.
3. Workflow advances records through review and history.
4. MMR consumes CAFM exports, applies business rules (documented in `module_mmr/CHARGEABLE_RULES.md`), and produces reports and optional automated email.
5. DocHub and BD add document-centric and commercial workflows on the same auth and data stack.

6. Operations and resilience

- Configuration is driven by `config.py` and environment variables (database, Redis, secrets, mail, MMR schedule overrides, etc.).
- Blueprint imports in `Injaaz.py` are often guarded so a failing module can be skipped without stopping the entire app; affected routes may show a clear error until fixed.

7. Further reading

Document	Contents
README.md	Quick start, tech stack summary
PROJECT_SCOPE_METHODS_AND_TECHNIQUES.md	Full scope, methods, libraries, blueprint map
SETUP.md	Full setup and environment
PROJECT_STRUCTURE.md	Directory layout

| [PROJECT_FLOW.md](#) | Application workflow |

| [module_mmr/CHARGEABLE_RULES.md](#) | MMR chargeable / non-chargeable rules |

| [Cloud / Render guides](#) | See repo root and docs/ for deployment runbooks |

This overview is maintained for stakeholders and new developers. Update it when major modules or URLs change.

Generated from markdown on 2026-04-09 15:04